

N-Gram Language Modeling and Evaluation

1. Overview and Objectives

The goal of this assignment is to implement and evaluate various **N-gram Language Models (LMs)** to understand the trade-offs between different N-gram orders, and the critical role of smoothing and backoff techniques in handling data sparsity.

2. Dataset

Source: <https://www.kaggle.com/datasets/aliakay8/penn-treebank-dataset>

- **Training Data:** Used for training the language model.
 - **Validation Data:** Used for tuning hyperparameters (e.g., smoothing parameters, interpolation weights).
 - **Test Data:** Used for final, unbiased performance evaluation.
-

3. Modeling and Evaluation Tasks

You will train and evaluate multiple models, varying the N-gram order, smoothing, and backoff/interpolation techniques. All evaluations must be performed using the **Perplexity (PP)** metric on the **Test Data**. Lower perplexity is better.

3.1. Maximum Likelihood Estimation (MLE) Models (N=1, 2, 3, 4)

Implement and evaluate N-gram LMs using Maximum Likelihood Estimation (MLE).

- **Handling Zeros:** The basic MLE models will have zero probabilities for unseen -grams. Perplexity is undefined in this case. Report the perplexity as a very large number, like INF for any model that encounters a zero probability on the test set. This highlights the necessity of smoothing.

3.2. Smoothing and Backoff Strategies

Implement the following strategies for (Trigram) models. Use the **Dev Data** to tune any necessary parameters before running the final evaluation on the **Test Data**.

3.2.1. Basic Smoothing

- **Add-1 Smoothing (Laplace):** Implement Add-1.
 - **Task:** Train and evaluate the model. Document the performance.

3.2.2. Backoff and Interpolation

- **Linear Interpolation:** Implement a linearly interpolated model that combines unigram, bigram, and trigram probabilities:
 - **Constraint:** Sum of the lambdas should be equal to 1. Try at least three combinations for the lambda values.
 - **Task:** Use the **Dev Data** to find the optimal weights that minimize perplexity. Use these optimal weights for the final evaluation on the **Test Data**.
 - **Stupid Backoff:** Implement the Stupid Backoff algorithm.
 - **Note:** The backoff continues down to the unigram. Set α (a common choice) or find an optimal α using the Dev Data. Document your choice.
-

4. Analysis and Discussion (Report Section)

Your report must include a detailed analysis of your results. Use markdown or clear formatting in your document.

4.1. Pre-processing and Vocabulary Decisions

- Explain your tokenization, sentence boundary and other preprocessing strategies.

4.2. Impact of N-gram Order

- Compare the perplexity results for all models.
- Discuss the trend you observe and explain this phenomenon in terms of the **Markov Assumption** and **Data Sparsity**.

4.3. Comparison of Smoothing/Backoff Strategies

- Compare the final perplexity scores of the best models.
- **Discussion:**
 - Why are the perplexity scores for the unsmoothed models so high/infinite, and how did smoothing correct this?
 - Compare the performance of the backoff/interpolation strategies. Which one performed the best and why do you think this is?

4.4. Qualitative Analysis (Generated Text)

- Implement a function to **generate a sequence of words** using the probability distribution of your **best performing model**.
- **Task:** Generate at least 5 distinct sentences and include them in your report.
- **Discussion:** How "fluent" or "human-like" does the generated text appear? Provide a brief explanation of how the model manages to generate these sequences.

5. Submission Checklist

- **Doc Report:**
 - All figures, tables, and results.
 - Detailed discussion answering all questions in Section 4.
- **Code:** [Link to your code repository (e.g., GitHub) or a separate code file].
 - Your code should be well-commented and runnable.
- **Readme File:** Describe how to run your code in detail.

6. Evaluation Criteria

Your submission will be graded on the following criteria:

- **Correctness (30%)**
- **Code Quality (30%)**
- **Efficiency and Edge Cases (20%)**
- **Analysis (20%)**